

扩散演化生成器：从零开始的实现与实验计划

面向普通本科毕业生的逐步执行手册：每一步做什么、为什么做、怎样验收

执行计划讨论稿

2026-07-15

目录

1	这份计划怎么使用	6
2	开始前必须统一的六条规则	6
2.1	规则 1：主线只做扩散演化生成器	6
2.2	规则 2：每条记录每轮只产生一个下一状态	6
2.3	规则 3：一轮内所有记录使用同一份旧残差	6
2.4	规则 4：先用 NumPy 和极小玩具数据验证，不先上 GPU	7
2.5	规则 5：每个随机实验都固定种子	7
2.6	规则 6：生成器运行期间不读取真实私有答案	7
3	最终仓库应该长什么样	7
4	阶段 0：先看懂原始 Diffusion Evolution 在做什么	9
4.1	目的	9
4.2	要做的工作	9
4.3	必须写清楚的五个问题	9
4.4	交付文件	9
4.5	验收条件	10
4.6	为什么通过后才能进入下一步	10
5	阶段 1：建立可复现的空仓库	10
5.1	目的	10
5.2	具体工作	10
5.3	交付文件	11
5.4	验收条件	11
5.5	最容易跑偏的地方	11

6 阶段 2：建立四状态玩具数据和查询评价器	11
6.1 目的	11
6.2 固定玩具数据	12
6.3 要实现的接口	12
6.4 必须测试	13
6.5 交付文件	13
6.6 验收条件	13
6.7 为什么这一步重要	13
7 阶段 3：实现残差和损失	13
7.1 目的	13
7.2 第一版固定规则	13
7.3 要实现的函数	14
7.4 必须测试	14
7.5 最容易跑偏的地方	14
8 阶段 4：写一个只用于测试的“真实替换收益”函数	14
8.1 目的	14
8.2 要做的工作	15
8.3 必须测试	15
8.4 交付文件	15
8.5 为什么这一步重要	15
9 阶段 5：实现最终个体适应度	15
9.1 目的	15
9.2 实现公式	16
9.3 玩具数据的标准答案	16
9.4 最关键的恒等式测试	16
9.5 要实现的接口	17
9.6 验收条件	17
9.7 常见错误	17
10 阶段 6：把适应度变成抽样概率	17
10.1 目的	17
10.2 工作 1：标准化	17
10.3 工作 2：温度控制	17
10.4 抽样频率测试	18
10.5 必须画两张图	18
10.6 验收条件	18

10.7 为什么还不能进入真实表	18
11 阶段 7: 实现简单、可检查的记录距离	18
11.1 目的	18
11.2 第一版只做二元或类别块	18
11.3 要实现的接口	19
11.4 必须测试	19
11.5 交付文件	19
11.6 常见跑偏	19
12 阶段 8: 每条记录只抽一个参考记录	19
12.1 目的	19
12.2 第一版使用全体记录作为参考池	19
12.3 接口	20
12.4 必须测试	20
12.5 必须输出的调试内容	20
12.6 通过标准	20
13 阶段 9: 实现一条记录的一步变化	21
13.1 目的	21
13.2 第一个版本关闭变异	21
13.3 接口	21
13.4 必须测试	21
13.5 为什么暂时不加变异	22
14 阶段 10: 实现一轮同步生成	22
14.1 目的	22
14.2 正确顺序	22
14.3 一个必须防止的错误	22
14.4 接口	22
14.5 一轮诊断必须包含	22
14.6 必须测试	23
15 阶段 11: 让玩具数据连续演化多轮	23
15.1 目的	23
15.2 实验设置	23
15.3 暂时使用的参数	24
15.4 每个种子必须画四张图	24
15.5 验收条件	24
15.6 如果失败, 按这个顺序排查	24

16 阶段 12：加入整代安全检查	25
16.1 目的	25
16.2 实现规则	25
16.3 接口	25
16.4 过冲测试	25
16.5 验收条件	25
17 阶段 13：加入合法变异，验证能发现缺失状态	26
17.1 目的	26
17.2 缺失状态实验	26
17.3 预期现象	26
17.4 实现要求	26
17.5 必须记录	26
17.6 验收条件	26
18 阶段 14：加入合法属性块和记录合法性	27
18.1 目的	27
18.2 人工合法规则	27
18.3 两种第一版实现方式	27
18.3.1 方式 A：把有关联的字段合并成一个块	27
18.3.2 方式 B：完整检查并回退	27
18.4 必须测试	27
18.5 最容易跑偏的地方	27
19 阶段 15：加入噪声容忍和查询权重	28
19.1 目的	28
19.2 实现顺序	28
19.2.1 第一步：只加入逆方差权重	28
19.2.2 第二步：加入有效残差	28
19.2.3 第三步：用 r_j 计算个体适应度	28
19.3 必须测试	28
19.4 消融	28
19.5 通过标准	29
20 阶段 16：把全体参考记录改成共享参考池	29
20.1 目的	29
20.2 实现方式	29
20.3 小数据对照实验	29
20.4 验收条件	29

21 阶段 17：第一个真实数据实验只用精确目标	30
21.1 目的	30
21.2 推荐数据	30
21.3 固定实验设置	30
21.4 必须比较的基础版本	30
21.5 指标	30
21.6 最低通过标准	31
22 阶段 18：接入固定的 DP 查询产物	31
22.1 目的	31
22.2 生成器只允许读取的文件	31
22.3 必须写的测试	31
22.4 评价流程	32
22.5 验收条件	32
23 阶段 19：做完整消融，回答方法到底靠什么工作	32
23.1 目的	32
23.2 必做消融	32
23.3 与定向演化的比较	32
24 阶段 20：最后才做性能优化	33
24.1 目的	33
24.2 优化顺序	33
24.3 每次优化的硬门槛	33
25 推荐时间表	33
26 每周汇报模板	34
27 每个阶段都要回答的五个问题	35
28 主线代码接口建议	35
29 必须长期保留的正确性测试	36
30 何时可以说“核心方法做成了”	37
31 现阶段明确不要做的事情	37
32 最后给执行者的一段话	38
33 参考材料	38

1 这份计划怎么使用

这不是一份“读完以后自由发挥”的建议，而是一套按顺序执行的任务清单。

每一个阶段都包含：

1. 这一阶段的目的；
2. 要做的具体工作；
3. 必须产生的文件和图；
4. 必须通过的测试；
5. 通过以后为什么才能进入下一阶段；
6. 最容易跑偏的地方。

原则是：

上一阶段的验收条件没有通过，不进入下一阶段；不靠长篇方案证明方法正确，只靠代码、数字、单元测试和最小实验逐步证明。

2 开始前必须统一的六条规则

2.1 规则 1：主线只做扩散演化生成器

主线不做：

按活跃查询寻找源记录

为查询构造目标记录

L0/L1/L2 定向候选

候选-查询二部图

逐候选动态贪心

二候选组合搜索

这些内容可以以后作为独立对照或混合增强，但不能在核心生成器尚未跑通时加入。

2.2 规则 2：每条记录每轮只产生一个下一状态

每条记录可以查看一个共享参考记录池，但最后只抽一个参考记录，只生成一个下一状态。

不能把“共享参考池”理解成“每条记录生成 512 个候选”。

2.3 规则 3：一轮内所有记录使用同一份旧残差

正确流程：

先固定 S_t

计算一次 $residual_t$

用 `residual_t` 计算全部记录适应度

全部记录同步产生下一状态

得到 S_{t+1}

再重新计算 `residual_{t+1}`

错误流程：

改一条记录

立刻更新残差

再决定下一条记录

后者会变成顺序定向搜索。

2.4 规则 4：先用 NumPy 和极小玩具数据验证，不先上 GPU

在 4 个状态、4 条记录上都不能完全验证的公式，换成 GPU 和 5 万条记录只会更难排错。

2.5 规则 5：每个随机实验都固定种子

必须保存：

seed

配置文件

代码提交号

数据版本

输出日志

否则无法判断结果变化来自方法还是随机性。

2.6 规则 6：生成器运行期间不读取真实私有答案

接入差分隐私实验以后，生成器只读取固定的带噪目标和噪声尺度。真实答案只能在算法运行结束后用于离线评价。

3 最终仓库应该长什么样

建议从第一天就建立下面的结构：

```
diffevo-table-generator/
|-- README.md
|-- pyproject.toml          # 或 requirements.txt
|-- configs/
|   |-- toy.yaml
|   |-- nltcs_exact.yaml
```

```

|   `-- nltcs_dp.yaml
|-- src/
|   `-- table_diffevo/
|       |-- schema.py
|       |-- queries.py
|       |-- objective.py
|       |-- fitness.py
|       |-- distance.py
|       |-- donor.py
|       |-- transition.py
|       |-- generator.py
|       |-- logging_utils.py
|       `-- random_utils.py
|-- tests/
|   |-- test_queries.py
|   |-- test_objective.py
|   |-- test_exact_replacement_gain.py
|   |-- test_fitness.py
|   |-- test_sampling.py
|   |-- test_distance.py
|   |-- test_transition.py
|   |-- test_generation.py
|   |-- test_legality.py
|   `-- test_no_private_answer_access.py
|-- notebooks/
|   |-- 00_original_diffevo.ipynb
|   |-- 01_four_state_toy.ipynb
|   `-- 02_missing_state.ipynb
|-- scripts/
|   |-- run_toy.py
|   |-- run_dataset.py
|   `-- summarize_runs.py
`-- outputs/
    `-- .gitkeep

```

第一版依赖尽量少:

```

numpy
pandas

```

```
scipy  
matplotlib  
pytest  
pyyaml
```

PyTorch、GPU 和复杂优化库等到正确性阶段完成后再加入。

4 阶段 0: 先看懂原始 Diffusion Evolution 在做什么

4.1 目的

在写表格代码前，先消除一个最容易导致整个项目跑偏的误解：

原始方法不是先随机生成许多候选再从中挑最好，而是先计算当前种群适应度，再根据适应度和邻近关系产生下一代。

4.2 要做的工作

1. 安装并运行官方 `diffusion-evolution` 示例；
2. 运行 `two-peak` 示例至少 3 个随机种子；
3. 每一轮保存：
 当前种群
 当前适应度
 生成后的下一代
4. 画出种群从初始随机状态逐渐向两个高适应度峰聚集的轨迹。

4.3 必须写清楚的五个问题

在 `notes/00_understanding.md` 中，用自己的话回答：

1. 当前种群是什么？
2. 适应度在什么时候计算？
3. 为什么单个个体的移动仍然有随机性？
4. 为什么整体种群会向高适应度区域移动？
5. 论文中的选择、邻近、变异分别对应代码中的什么？

4.4 交付文件

```
notebooks/00_original_diffevo.ipynb  
notes/00_understanding.md  
outputs/stage00/trajectory_seed0.png
```

outputs/stage00/trajectory_seed1.png

outputs/stage00/trajectory_seed2.png

4.5 验收条件

- 示例能从头运行;
- 三个种子都能看到种群朝高适应度区域聚集;
- 能口头解释“随机结果”和“有方向的概率分布”为什么不矛盾;
- 不再使用“原方法就是纯随机候选”这样的表述。

4.6 为什么通过后才能进入下一步

如果没有理解原始方法, 后面很容易把表格生成器写成候选搜索器。阶段 0 是方法边界测试, 不是形式任务。

5 阶段 1: 建立可复现的空仓库

5.1 目的

保证以后每一次结果都能重现, 并让单元测试从第一天开始存在。

5.2 具体工作

1. 建立上述目录;
2. 建立虚拟环境;
3. 安装依赖;
4. 写统一随机种子函数:

```
def set_seed(seed: int) -> np.random.Generator:
    return np.random.default_rng(seed)
```

5. 写第一个测试:

```
def test_seed_is_reproducible():
    rng1 = np.random.default_rng(0)
    rng2 = np.random.default_rng(0)
    assert np.array_equal(rng1.integers(0, 100, 20),
                           rng2.integers(0, 100, 20))
```

6. 建立最小配置文件:

```
seed: 0
num_rounds: 100
noise_tolerance_kappa: 0.0
```

```
temperature_start: 2.0
temperature_end: 0.7
neighbor_radius_start: 0.8
neighbor_radius_end: 0.15
update_rate_start: 0.10
update_rate_end: 0.02
copy_rate_start: 0.50
copy_rate_end: 0.10
mutation_rate_start: 0.0
mutation_rate_end: 0.0
```

5.3 交付文件

```
README.md
pyproject.toml 或 requirements.txt
configs/toy.yaml
src/table_diffevo/random_utils.py
tests/test_random_utils.py
```

5.4 验收条件

```
pytest -q
```

必须全绿。

5.5 最容易跑偏的地方

- 不要在这个阶段开始写算法;
- 不要先搭复杂训练框架;
- 不要先接真实数据;
- 不要把输出手工复制到不同目录, 所有路径由配置统一管理。

6 阶段 2: 建立四状态玩具数据和查询评价器

6.1 目的

先保证“记录是否满足查询”这件最基础的事情完全正确。

6.2 固定玩具数据

两个二元属性:

$$A, B \in \{0, 1\}.$$

四条记录:

row 0: A=0, B=0

row 1: A=0, B=1

row 2: A=1, B=0

row 3: A=1, B=1

三个查询:

q1: A = 1

q2: B = 1

q3: A = 1 and B = 1

当前答案必须是:

$$q(S) = (2, 2, 1).$$

目标设为:

$$y = (3, 1, 1).$$

6.3 要实现的接口

```
class Query:
    def evaluate_row(self, row) -> int:
        ...

def evaluate_rows(rows, queries) -> np.ndarray:
    """ 返回 shape=[N, m] 的 0/1 矩阵。 """

def query_answers(query_matrix: np.ndarray) -> np.ndarray:
    return query_matrix.sum(axis=0)
```

6.4 必须测试

- 每条记录对每个查询的 0/1 结果;
- 查询矩阵形状为 (4, 3);
- 列和严格等于 (2,2,1);
- 查询实现不能依赖目标值;
- 输入顺序变化时, 查询答案不变。

6.5 交付文件

```
src/table_diffevo/queries.py
tests/test_queries.py
notebooks/01_four_state_toy.ipynb
```

6.6 验收条件

每个数字都能手算对照, 不能只看“程序没有报错”。

6.7 为什么这一步重要

后面所有残差、适应度和演化方向都依赖这个 0/1 矩阵。查询评价错一位, 后面的数学全部失去意义。

7 阶段 3: 实现残差和损失

7.1 目的

固定符号和尺度, 避免最常见的“方向反了”和“计数比例混用”。

7.2 第一版固定规则

只用计数, 不用比例。

$$e = y - q(S).$$

玩具数据中:

$$e = (1, -1, 0).$$

第一版所有查询权重为 1, $\kappa = 0$ 。损失:

$$L(S) = \frac{1}{2} \sum_j e_j^2 = 1.$$

7.3 要实现的函数

```
def compute_raw_residual(target, current_answer):
    return target - current_answer

def compute_active_residual(raw_residual, sigma, kappa):
    return np.sign(raw_residual) * np.maximum(
        np.abs(raw_residual) - kappa * sigma,
        0.0,
    )

def workload_loss(raw_residual, sigma, weights, kappa):
    active = compute_active_residual(raw_residual, sigma, kappa)
    return 0.5 * np.sum(weights * active ** 2)
```

7.4 必须测试

1. 玩具残差是 (1, -1, 0);
2. $\kappa = 0$ 时损失等于 1;
3. 残差正负号符合“目标减当前”;
4. 若当前答案等于目标, 损失为 0;
5. 任何输入不允许出现 NaN。

7.5 最容易跑偏的地方

- 把残差写成 `current - target`;
- 查询用比例, 目标用计数;
- 提前加入复杂高斯容忍, 让基础公式难以验证。

8 阶段 4: 写一个只用于测试的“真实替换收益”函数

8.1 目的

建立一个绝对可靠但很慢的检查器。以后适应度公式是否正确, 都用它验证。

这个函数不是最终生成算法，也不是要转向定向搜索。它只是单元测试的“标准答案”。

8.2 要做的工作

对每一个源记录 x_i 和每一个目标状态 z :

1. 复制当前小表;
2. 把第 i 条记录替换为 z ;
3. 完整重算查询答案和损失;
4. 返回:

$$G(i, z) = L(S) - L(S^{i \rightarrow z}).$$

再实现公式版:

$$G(i, z) = e^\top W(a(z) - a(x_i)) - \frac{1}{2} \|a(z) - a(x_i)\|_W^2.$$

8.3 必须测试

对 4 个源记录和 4 个目标状态，共 16 种替换:

暴力重算收益 == 公式收益

绝对误差小于 10^{-12} 。

8.4 交付文件

```
src/table_diffevo/objective.py
tests/test_exact_replacement_gain.py
```

8.5 为什么这一步重要

没有这个检查器，适应度算错时很难判断到底是查询、残差、符号还是公式出了问题。

9 阶段 5: 实现最终个体适应度

9.1 目的

把“某条记录改成某个目标的收益”转成“某种记录形态是否值得变得更常见”的个体适应度。

9.2 实现公式

当前平均查询贡献:

$$\bar{a} = \frac{1}{N} \sum_i a(x_i).$$

对记录 z :

$$F(z) = e^\top W(a(z) - \bar{a}) - \frac{1}{2} \|a(z) - \bar{a}\|_W^2.$$

第一版用 e , 不要先用噪声容忍后的残差。

9.3 玩具数据的标准答案

状态	适应度
00	-0.28125
01	-1.28125
10	0.71875
11	-0.53125

10 必须最高。

9.4 最关键的恒等式测试

对每个状态 z , 计算:

$$\bar{G}(z) = \frac{1}{N} \sum_i G(i, z).$$

程序必须验证:

$$\bar{G}(z) - F(z)$$

对所有 z 都是同一个常数。

在本玩具数据中, 这个常数应为:

$$-0.34375.$$

9.5 要实现的接口

```
def individual_fitness(
    query_matrix: np.ndarray,
    residual: np.ndarray,
    query_weights: np.ndarray,
) -> np.ndarray:
    ...
```

9.6 验收条件

- 四个标准数值完全匹配;
- 10 的适应度最高;
- 平均真实替换收益与适应度只差同一个常数;
- 把残差全部设为 0 后, 适应度不再表现出某个查询方向的偏好。

9.7 常见错误

- 没有减去种群平均贡献 $\bar{a}$;
- 二次项符号写成加号;
- 直接给低适应度截断为 0, 导致排序关系变化;
- 用更新后的表计算一部分适应度, 造成同一轮适应度不一致。

10 阶段 6: 把适应度变成抽样概率

10.1 目的

确认“适应度高的状态更容易被抽到”, 同时保留随机性。

10.2 工作 1: 标准化

```
z = (fitness - fitness.mean()) / (fitness.std() + 1e-12)
```

10.3 工作 2: 温度控制

第一版先不考虑记录距离:

$$p_i \propto \exp(z_i/\tau).$$

10.4 抽样频率测试

在前面的四状态例子中，如果直接用未标准化的原始适应度且 $\tau = 1$ ，理论概率约为：

状态	概率
00	0.2056
01	0.0756
10	0.5587
11	0.1601

抽样 100000 次，经验频率应接近理论概率。允许误差可以设为 0.01。

10.5 必须画两张图

1. 不同温度下四个状态的理论概率；
2. 理论概率与 100000 次经验频率的对照。

10.6 验收条件

- 高频状态确实更常被抽到；
- 温度越大，概率越平；
- 温度越小，概率越集中；
- 不出现溢出、NaN 或概率和不为 1。

10.7 为什么还不能进入真实表

此时只证明“适应度能变成正确的随机偏好”，还没有实现“相似记录更容易相互学习”。

11 阶段 7: 实现简单、可检查的记录距离

11.1 目的

让记录主要向相似的高适应度记录学习，防止全表一步坍缩。

11.2 第一版只做二元或类别块

归一化 Hamming 距离：

$$d(x, z) = \frac{\text{不同属性块的数量}}{\text{可修改属性块总数}}$$

11.3 要实现的接口

```
def pairwise_block_distance(rows, donor_rows, schema):
    """ 返回 shape=[N, M], 所有值在 [0, 1]。 """
```

11.4 必须测试

- $d(x, x) = 0$;
- $d(x, z) = d(z, x)$;
- 所有值在 0 到 1;
- 只差一个块的距离小于差两个块;
- 不可修改属性不进入距离;
- 两个必须共同合法的字段作为一个块计算。

11.5 交付文件

```
src/table_differvo/schema.py
src/table_differvo/distance.py
tests/test_distance.py
```

11.6 常见跑偏

不要在这里写“根据某个查询的条件计算距离”。距离只描述记录之间有多相似，不知道哪个查询当前有误差。

12 阶段 8: 每条记录只抽一个参考记录

12.1 目的

完成扩散演化最核心的概率步骤：高适应度且相近的记录更容易成为参考。

12.2 第一版使用全体记录作为参考池

对当前记录 x_i 和参考记录 z_k ：

$$\ell_{ik} = \frac{f_k^{\text{standard}}}{\tau} - \frac{d(x_i, z_k)^2}{2h^2}.$$

对每一行做 softmax，只抽一个索引。

12.3 接口

```
def sample_one_donor_per_row(
    rows,
    donor_rows,
    donor_standardized_fitness,
    temperature,
    neighbor_radius,
    rng,
) -> np.ndarray:
    """ 返回长度  $N$  的 donor 索引; 每条记录只有一个。 """
```

12.4 必须测试

1. 适应度相同、距离不同时, 更近的记录更常被选;
2. 距离相同、适应度不同时, 高适应度记录更常被选;
3. h 大时, 距离影响减弱;
4. h 小时, 几乎只选近邻;
5. 每条记录输出恰好一个索引;
6. 同一个随机种子输出完全相同。

12.5 必须输出的调试内容

对玩具数据的每一条记录打印:

当前记录

每个参考记录的适应度

每个参考记录的距离

最终 `logit`

最终概率

抽到的参考记录

12.6 通过标准

看到这些数字时, 能够不用代码术语解释为什么某个参考记录概率更大。

13 阶段 9: 实现一条记录的一步变化

13.1 目的

把“抽到一个参考记录”变成“当前记录向它靠近一步”，而不是直接整行复制。

13.2 第一个版本关闭变异

先设：

```
mutation_rate = 0
```

只实现：

1. 以概率 ρ 参与更新；
2. 对每个不同属性块，以概率 η 复制参考块；
3. 生成后检查合法；
4. 非法则保留原记录。

13.3 接口

```
def evolve_one_row(  
    row,  
    donor,  
    update_rate,  
    copy_rate,  
    mutation_rate,  
    schema,  
    rng,  
):  
    ...
```

13.4 必须测试

- `update_rate=0` 时永远不变；
- `update_rate=1, copy_rate=0` 时不变；
- `update_rate=1, copy_rate=1` 时复制所有允许复制的不同块；
- 不可修改块永远不变；
- 输出永远合法；
- 不改变输入数组本身；
- 固定种子时结果可复现。

13.5 为什么暂时不加变异

先证明“选择参考记录并向它靠近”这条主路径完全正确。变异会增加随机分支，使排错更困难。

14 阶段 10: 实现一轮同步生成

14.1 目的

把前面模块连接成一轮完整的表格扩散演化。

14.2 正确顺序

1. 固定当前表 `S_t`
2. 计算一次查询矩阵和残差
3. 计算全部记录适应度
4. 为全部记录抽参考记录
5. 为全部记录生成下一状态
6. 所有状态组合成 `S_proposal`
7. 再计算 `S_proposal` 的查询答案

14.3 一个必须防止的错误

下面的写法禁止出现：

```
for i in range(N):
    修改第 i 条记录
    立刻更新 residual
    用新 residual 处理第 i+1 条记录
```

这会把同步种群演化改成顺序贪心。

14.4 接口

```
def propose_generation(S, targets, queries, config, rng):
    return proposal_S, diagnostics
```

14.5 一轮诊断必须包含

```
old_answers
old_residual
old_loss
```

```
fitness
chosen_donor_ids
chosen_donor_distances
proposed_change_mask
new_answers
new_residual
new_loss
```

14.6 必须测试

- 同一轮所有适应度都来自同一个旧表;
- 输入行顺序改变后, 统计结果不应系统性改变;
- 同一随机种子完全可复现;
- 输出行数与输入相同;
- 输出合法率 100%。

15 阶段 11: 让玩具数据连续演化多轮

15.1 目的

第一次验证“多轮以后整张表的统计方向正确”。

15.2 实验设置

使用更大的玩具表, 例如 1000 条记录, 由四种状态构成:

00: 200 条
01: 400 条
10: 300 条
11: 100 条

当前:

A=1 共 400 条
B=1 共 500 条

目标可设为:

A=1 目标 600 条
B=1 目标 300 条
A=1 且 B=1 目标 100 条

直观上应该让 10 增多、01 减少。

15.3 暂时使用的参数

```
temperature: 2.0 -> 0.7
neighbor radius: 0.8 -> 0.15
update rate: 0.10 -> 0.02
copy rate: 0.50 -> 0.10
mutation rate: 0
rounds: 100
```

15.4 每个种子必须画四张图

1. 总损失随轮数;
2. 每个查询残差随轮数;
3. 四种记录状态的比例随轮数;
4. 每轮实际变化记录比例。

至少运行 20 个随机种子。

15.5 验收条件

- 大多数种子中总损失明显低于初始值;
- 中位数损失随轮数总体下降;
- 10 的比例总体上升;
- 01 的比例总体下降;
- 结果没有迅速坍缩为单一状态;
- 单个种子偶尔波动可以接受, 但不能所有种子都没有方向。

15.6 如果失败, 按这个顺序排查

1. 重新检查残差符号;
2. 重新检查四状态适应度标准值;
3. 把距离项关掉, 只看适应度抽样;
4. 把温度调低, 观察方向是否出现;
5. 把更新率调低, 观察是否是过冲;
6. 打印一轮所有概率, 不要直接换方法。

16 阶段 12: 加入整代安全检查

16.1 目的

解决几万条记录同步使用同一份旧残差时可能发生的过冲。

16.2 实现规则

1. 产生完整下一代提案;
2. 如果提案损失下降, 接受;
3. 如果提案损失上升:
 - 将本轮更新率 ρ 减半;
 - 使用同一旧表和旧残差重新抽一代;
 - 最多重试 3 到 5 次;
 - 仍失败则本轮保持原表。

16.3 接口

```
def propose_with_generation_guard(...):  
    return accepted_S, diagnostics
```

16.4 过冲测试

构造一个只有单个查询的表:

- 当前缺 20 条满足记录;
- 设置很大的 `update_rate=1`;
- 无保护时大量记录同时变化, 容易超过目标;
- 加保护后应自动降低更新率。

16.5 验收条件

- 所有被接受的整代损失都不高于旧表;
- 日志记录每轮重试次数和最终使用的更新率;
- 保护机制不逐条查看或排序记录;
- 保护机制只缩小整代步幅。

17 阶段 13: 加入合法变异, 验证能发现缺失状态

17.1 目的

证明生成器不只会复制当前已有记录, 还能发现初始种群中没有的新模式。

17.2 缺失状态实验

仍使用 $A, B \in \{0, 1\}$, 但初始表中完全删除 11。

加入一个目标查询:

q3: $A=1$ and $B=1$, 目标明显大于 0

比较:

1. `mutation_rate=0`;
2. `mutation_rate>0`。

17.3 预期现象

- 无变异时, 如果复制规则无法组合出 11, 目标难以达到;
- 有变异时, 11 偶尔出现;
- 一旦 11 在当前残差下适应度较高, 它会更常被选为参考, 数量逐渐增加。

17.4 实现要求

第一版每条参与更新的记录最多变异一个属性块。

```
def mutate_one_block(row, schema, rng):  
    ...
```

17.5 必须记录

每轮变异次数

变异后合法次数

新状态首次出现轮次

新状态后续数量变化

17.6 验收条件

- 关闭变异时缺失状态不会凭空出现;
- 打开变异后, 多数种子能在合理轮数内出现缺失状态;
- 新状态出现后, 如果它适应度高, 数量会通过选择机制增长;

- 变异不会造成非法记录。

18 阶段 14: 加入合法属性块和记录合法性

18.1 目的

解决“每个单字段合法，但组合起来不合法”的问题，同时不回到定向候选构造。

18.2 人工合法规则

设计一个简单约束，例如：

如果 A=1，则 B 必须等于 1

合法状态只有：

00, 01, 11

10 非法。

18.3 两种第一版实现方式

18.3.1 方式 A：把有关联的字段合并成一个块

块 AB 的合法值 = {00, 01, 11}

复制和变异都以整个块为单位。

18.3.2 方式 B：完整检查并回退

先产生记录，再调用 `is_valid`；非法则重试或保持原记录。

优先使用方式 A，因为从源头减少非法提案。

18.4 必须测试

- 初始化合法率 100%；
- 任意一轮输出合法率 100%；
- 不可修改字段变化率 0%；
- 变异只从合法块状态中抽样；
- 距离按块计算，不把一个模板变化重复计算多次。

18.5 最容易跑偏的地方

不要为每个查询写一套专门的“合法修复器”。合法规则属于数据模式，不属于某个查询的残差方向。

19 阶段 15: 加入噪声容忍和查询权重

19.1 目的

防止生成器追着高斯噪声反复微调。

19.2 实现顺序

19.2.1 第一步: 只加入逆方差权重

$$w_j = \frac{1}{\sigma_j^2 + \sigma_{\min}^2}.$$

归一化到平均为 1。

19.2.2 第二步: 加入有效残差

$$r_j = \text{sign}(e_j) \max(|e_j| - \kappa\sigma_j, 0).$$

19.2.3 第三步: 用 r_j 计算个体适应度

监控损失也使用同一容忍规则。

19.3 必须测试

1. $|e_j| < \kappa\sigma_j$ 时, $r_j = 0$;
2. $e_j > \kappa\sigma_j$ 时, 方向为正;
3. $e_j < -\kappa\sigma_j$ 时, 方向为负;
4. 小方差查询权重更大;
5. 所有权重归一化后平均为 1;
6. 一个已进入容忍区的查询不再通过残差线性项给出“增加或减少”的方向; 二次项仍可用于防止一步破坏过大。

19.4 消融

至少比较:

`kappa = 0`

`kappa = 1`

`kappa = 1.645`

19.5 通过标准

加入噪声容忍以后, 后期震荡应减少; 但不能因为容忍过大而使大量明显残差被忽略。

20 阶段 16: 把全体参考记录改成共享参考池

20.1 目的

把两两计算从 $O(N^2)$ 降为近似 $O(NM)$, 使几万条记录可运行。

20.2 实现方式

参考池大小 M 先取 256 或 512:

75% 均匀随机记录

25% 当前适应度最高记录

接口:

```
def build_donor_pool(fitness, pool_size, top_fraction, rng):  
    ...
```

20.3 小数据对照实验

在一个可以使用全体参考记录的小数据集上比较:

1. 全体参考;
2. $M = 64$;
3. $M = 128$;
4. $M = 256$;
5. $M = 512$ 。

比较:

最终损失

达到同一损失所需轮数

运行时间

参考距离分布

结果方差

20.4 验收条件

- 参考池版本仍保留明显的适应度方向;
- M 增大时结果逐渐接近全体参考版本;

- 运行时间随 N 近似线性增长;
- 不把参考池中的每一条记录都生成候选。

21 阶段 17: 第一个真实数据实验只用精确目标

21.1 目的

先验证生成器在真实表格结构上能工作, 不让 DP 噪声、查询投影和预算分配同时干扰排错。

21.2 推荐数据

优先选择二元属性数据, 例如 NLTCs。原因:

- 记录表示简单;
- 合法性问题少;
- 查询评价容易核对;
- 适合先验证扩散演化核心。

21.3 固定实验设置

1. 固定训练/评价划分;
2. 生成固定数量的 1-way 和 2-way 查询;
3. 用精确查询答案作为目标;
4. 使用同一张初始合成表;
5. 运行至少 5 个种子。

21.4 必须比较的基础版本

1. **Random**: 不使用适应度, 只做同样幅度的随机变化;
2. **Fitness only**: 使用适应度, 但不使用距离;
3. **Fitness + distance**: 完整参考抽样;
4. **Full generator**: 再加变异、计划和整代保护。

21.5 指标

训练查询 RMSE

留出查询 RMSE

最大绝对误差

最终损失

达到固定损失的时间

每秒处理记录数

记录变化率

参考概率熵

新状态出现率

21.6 最低通过标准

完整生成器至少满足一项：

- 相同时间下训练查询误差更低；
- 相同训练误差下用时更少；
- 留出查询明显优于 Random；
- 比 Fitness only 更能保持多样性。

如果四个版本没有清晰差异，不进入 DP 实验，先定位核心模块是否真正起作用。

22 阶段 18: 接入固定的 DP 查询产物

22.1 目的

在不改变生成器结构的前提下，把输入从精确目标换成带噪目标。

22.2 生成器只允许读取的文件

queries.json

projected_targets.npy

noise_std.npy

initial_synthetic_table.npy 或 parquet

schema.json

禁止在运行路径中读取：

true_answers.npy

private_train.csv

任何真实误差反馈

22.3 必须写的测试

test_no_private_answer_access.py:

- 把真实答案文件移走后，生成器仍能完整运行；
- 代码中不允许有读取真实答案的接口；
- 真实指标只在单独的评价脚本中计算。

22.4 评价流程

生成器运行: 只看带噪目标

运行结束: 保存最终表

独立评价脚本: 读取真实答案, 计算指标

22.5 验收条件

- 同一固定 transcript 和随机种子可复现;
- 生成器内部损失按带噪目标下降;
- 真实效用只作外部报告, 不反向调节当前运行。

23 阶段 19: 做完整消融, 回答方法到底靠什么工作

23.1 目的

避免最后只得到一个复杂系统, 却不知道哪个模块有用。

23.2 必做消融

版本	回答的问题
无适应度	如果完全随机, 会怎样
只有适应度、无距离	邻近限制是否防止坍缩
适应度 + 距离	扩散演化核心是否成立
无二次惩罚项	步幅惩罚是否减少过冲
无变异	缺失状态能否被发现
无整代保护	同步更新是否会过冲
固定参数	逐步减小范围和步幅是否有帮助
全体参考 vs 参考池	近似是否损失太多
$\kappa = 0, 1, 1.645$	噪声容忍如何影响拟合与过拟合

23.3 与定向演化的比较

定向演化可以作为外部基线, 但比较重点应是:

相同时间预算下的误差

达到同一误差所需时间

每秒完成的记录更新数

内存

多样性
留出查询

不要只比较最终训练误差，因为扩散演化的核心动机是批量化和速度-效用折中。

24 阶段 20：最后才做性能优化

24.1 目的

在数学和实验都正确以后，再提高速度。

24.2 优化顺序

- 1. 缓存当前每条记录的查询贡献；
- 2. 只重算发生变化记录的贡献；
- 3. 压缩重复记录为“唯一状态 + 计数”；
- 4. NumPy 批量化；
- 5. PyTorch CPU；
- 6. PyTorch GPU；
- 7. 分块计算记录-参考距离；
- 8. 更快的近邻检索。

24.3 每次优化的硬门槛

优化版必须与慢速基线对照：

- 查询答案一致
- 损失一致
- 固定随机流下抽样概率一致
- 合法率一致

性能优化不能改变方法定义。

25 推荐时间表

时间只是参考，验收门槛比日期更重要。

周	主要阶段	本周结束必须拿到的结果
第 1 周	阶段 0-3	原始示例复现；四状态查询、残差、损失全对
第 2 周	阶段 4-6	真实替换收益检查器；适应度恒等式；抽样频率图
第 3 周	阶段 7-10	距离、参考抽样、一条记录变化、一轮同步生成

周	主要阶段	本周结束必须拿到的结果
第 4 周	阶段 11-12	多轮玩具收敛图；整代过冲保护
第 5 周	阶段 13-15	缺失状态、合法块、噪声容忍
第 6 周	阶段 16	共享参考池与规模实验
第 7 周	阶段 17	第一个公开数据精确目标实验
第 8 周	阶段 18-19	固定 DP transcript 和核心消融
后续	阶段 20	性能优化、更多数据集、论文实验

如果某周的验收没有通过，就顺延，不应通过增加新模块掩盖旧问题。

26 每周汇报模板

每周只需要按下面格式汇报，避免又写成长篇但无法验证的方案。

1. 本周完成了哪个阶段：
2. 这一阶段的目的是什么：
3. 新增了哪些函数：
4. 新增了哪些单元测试：
5. pytest 结果：
6. 最关键的手算对照数字：
7. 生成了哪些图：
8. 哪个结果与预期不一致：
9. 目前判断最可能的原因：
10. 下一步只做什么：

开会时优先看：

 - 测试；
 - 数字；
 - 曲线；

- 代码接口;

不优先看新的大而全方法图。

27 每个阶段都要回答的五个问题

无论做到哪一步，都必须能用普通话回答：

1. 这一步的输入是什么？
2. 这一步的输出是什么？
3. 为什么需要这一步？
4. 用哪个最小例子证明它正确？
5. 如果这一步错了，后面的结果会出现什么现象？

如果只能背公式，不能回答这五个问题，说明还没有真正理解。

28 主线代码接口建议

最终核心接口尽量保持简单：

```
def evaluate_rows(rows, queries) -> np.ndarray:
    """ 每条记录对每个查询的 0/1 贡献。 """

def compute_residual(target, current_answer) -> np.ndarray:
    """ 目标减当前答案。 """

def compute_individual_fitness(
    query_matrix,
    active_residual,
    query_weights,
) -> np.ndarray:
    """ 每条当前记录的繁殖价值。 """

def sample_one_donor_per_row(
    rows,
    donor_rows,
    donor_fitness,
    temperature,
```

```

        neighbor_radius,
        rng,
    ) -> np.ndarray:
        """ 每条记录只返回一个参考记录编号。 """

```

```

def evolve_all_rows_once(
    rows,
    chosen_donors,
    update_rate,
    copy_rate,
    mutation_rate,
    schema,
    rng,
) -> np.ndarray:
    """ 每条记录只生成一个下一状态。 """

```

```

def run_generator(
    initial_table,
    queries,
    targets,
    noise_std,
    schema,
    config,
):
    """ 多轮同步演化，输出最优完整表和诊断。 """

```

如果一个模块开始出现“active_query_id、source_record、target_record、candidate_list、greedy_select”等主线字段，就要停下来检查是否已经转回定向搜索。

29 必须长期保留的正确性测试

下面这些测试一旦写好，不允许为了让新代码通过而删除：

四状态查询矩阵标准答案

残差符号标准答案

16 种替换的暴力收益 == 公式收益

个体适应度 == 平均替换收益 + 同一常数

抽样经验频率接近理论概率

距离自反、对称、范围正确

不可修改块永远不变

一轮内所有记录使用同一旧残差

固定种子可复现

每轮行数不变

合法率 100%

整代保护后已接受损失不升

生成器不读取真实私有答案

30 何时可以说“核心方法做成了”

只有同时满足以下条件，才算核心方法完成：

1. 四状态玩具的全部数学测试通过；
2. 多轮种群在大多数随机种子中朝正确方向演化；
3. 缺失状态能通过变异出现并被高适应度放大；
4. 合法规则下非法记录率为 0；
5. 共享参考池版本在真实数据规模可运行；
6. 完整方法明显优于无适应度随机基线；
7. 生成器不需要逐查询构造目标记录；
8. 每条记录每轮只产生一个下一状态；
9. 同步更新不会因过冲长期发散；
10. 固定 DP transcript 上可以重复得到同样的统计结果。

31 现阶段明确不要做的事情

为了避免再次跑偏，在阶段 17 以前不要做：

为每个查询寻找最近源记录

按残差符号构造进入/退出查询的目标行

L0/L1/L2 候选扩展

半空间最小跨越搜索

候选-查询二部图

动态边际收益贪心

二候选联合恢复

堆、lazy greedy、增量候选图

复杂连续属性专用算法

完整协方差和大规模理论证明

这些并不是永远不能研究，而是不能在扩散演化核心尚未通过最小实验时抢占主线。

32 最后给执行者的一段话

这项工作的难点不在于一次想出一个看起来很完整的方案，而在于把下面这条链逐个证明：

查询答案算对

- > 残差方向算对
- > 个体适应度算对
- > 适应度真的改变抽样概率
- > 相似度真的限制学习范围
- > 每条记录只生成一个下一状态
- > 一整代在平均上朝正确方向移动
- > 多轮以后查询误差下降
- > 变异能补充缺失模式
- > 合法性和规模化仍然成立

每一步都不难，但必须按顺序做。只要这条链完整跑通，后面再讨论更复杂的合法模板、连续属性、混合定向增强和论文创新，才有可靠基础。

33 参考材料

[1] Yanbo Zhang, Benedikt Hartl, Hananel Hazan, Michael Levin. *Diffusion Models are Evolutionary Algorithms*. ICLR 2025.

[2] Zhangyanbo. *diffusion-evolution* official code repository.

[3] 配套文档：《基于查询残差的表格扩散演化生成器：完整方法设计》。

[4] 内部对照材料：《高斯噪声感知的残差驱动合法候选与冲突感知动态选择方法：最终统一实现设计版》。